

Titanic 生存预测模型学习笔记

—— 基于 PyTorch 的二分类模型

一、任务简介

本次要使用 Titanic 数据集完成一个二分类任务，目标是根据乘客的相关信息预测其是否能够生还。其中“是否生还”只有两种结果，所以属于典型的二分类问题。整个实验使用 PyTorch 自己定义神经网络，并完成数据处理、模型训练、模型保存和新样本预测。

下面的内容按照两个源代码文件的书写思路进行记录。第一部分对应 tainiTrain.py，主要讲模型是怎样训练出来的；第二部分对应 predict.py，主要讲训练好的模型是怎样被重新加载并用于预测新乘客的，以及如何构建互动体验的。

二、tainiTrain.py：训练模型

1. 导入必要的库并固定随机种子

我在程序开始时首先导入 Pandas 和 PyTorch。Pandas 主要负责读取和处理 Titanic 表格数据，PyTorch 负责构建一个二分类神经网络。

我接着把随机种子设置为 42。因为在数据集划分过程中使用了随机打乱，如果每次运行都完全随机，那么每次得到的训练集和测试集可能不同，最后的准确率也可能发生变化。固定随机种子以后，随机过程可以尽量保持一致，这对于实验复现很有效。

2. 读取原始数据

接下来程序读取 train.csv。读取以后，整个 Titanic 数据被保存成一个 Pandas DataFrame，也就是一张可以通过代码操作的表格。表格中的每一行代表一名乘客，每一列代表一种信息。

3. 进行特征工程

原始数据中的一些字段并不太适合直接交给神经网络，所以应该先根据已有数据构造一些新的特征，即一些可能对预测真正有帮助的信息。

首先从 Cabin 中取出最前面的字母作为 Deck，用来表示乘客所在的甲板。因为我认为后面的数字对模型的学习并没有多大帮助。如果 Cabin 缺失，就统一记为 Unknown，这样虽然没有完全恢复具体的船舱位置，但至少保留了一部分甲板信息。

然后根据 SibSp 和 Parch 计算 FamilySize（加上乘客本人），即家庭人数，因为我认为这个数据更直接简洁。接着根据 FamilySize 是否等于 1 来生成 IsAlone，用 1 表示独自乘船，用 0 表示不是独自乘船。

我接着从 Name 中提取称谓 Title，例如 Mr、Mrs、Miss 等。因为我认为名字本身对生存概率影响不大，而不同称谓可能和乘客的年龄、性别以及社会身份有一定关系，因此把它单独提取出来作为一个分类特征。

最后计算 FarePerPerson，也就是平均每个人承担的票价。这样可以避免模型只看总票价，便于模型学习理解。

4. 确定预测目标并划分数据

接下来程序把 Survived 单独拿出来作为目标变量，也就是模型最终需要预测的结果。Survived 为 1 表示生还，为 0 表示没有生还。剩下的字段作为输入特征。

数据随后按照 80% 和 20% 的比例划分为训练集和测试集。并且在划分之前先随机打乱数据，可以减少原始数据排列顺序对结果造成的影响。

在这里我注意到一个细节：后面很多预处理参数，例如年龄中位数、登船港口众数、标准化的平均值和标准差，都应该只从训练集计算。这样可以避免测试集的信息提前泄露。

5. 处理缺失值

Titanic 数据中 Age 和 Embarked 等字段存在缺失。对于 Age，程序使用训练集年龄的中位数进行填充。（我记得中位数对少数极端值没有那么敏感）

对于 Embarked，也就是登船港口，我使用训练集中出现次数最多的类别（众数）进行填充。我还注意到一个细节，测试集中的缺失值没有重新计算自己的中位数或众数，而是直接使用训练集计算出的结果。这样才能保证训练和测试使用同一套规则。

6. 删除不直接使用的原始字段

PassengerId 只是乘客编号，本身没有实际的预测含义。Name 和 Cabin 中有用的信息前面已经被提取成 Title 和 Deck，所以这里也删掉。Ticket 也因被工程处理了而删除。

经过这一步以后，数据中保留下来的主要是比较适合进一步处理的数值特征和分类特征。

7. 处理分类特征

Sex、Embarked、Deck 和 Title 都属于分类变量。神经网络不能直接把这些英文单词当作普通数字来理解，因此需要把它们转换成数值。

这里采用的是**独热编码**。比如性别只有 male 和 female 两种情况，就可以转换成两个 0/1 特征。某个乘客是 male 时，对应 male 的位置为 1，female 的位置为 0。其他分类变量也是类似的处理方式。

我还特别**保存了训练集中的类别信息**。测试集如果出现训练阶段没有出现过的类别，就不会随意增加新列，而是按照训练时确定的特征结构进行处理，最后再调整测试集的列顺序，使训练集和测试集拥有**完全相同的输入特征**。

8. 对数值特征进行标准化

经过分类变量处理以后，还需要处理数值变量。当前模型中的 Pclass、Age、SibSp、Parch、Fare、FamilySize、IsAlone 和 FarePerPerson **数值范围并不完全一样**。例如 Fare 的数值可能比较大，而 IsAlone 只有 0 和 1。

因此我对这些数值特征进行标准化，把它们转换到**比较接近的尺度**。这里最重要的不是公式本身，而是**参数来源**。平均值和标准差全部根据训练集计算，测试集只使用这些**已经计算好的参数**。以后 predict.py 预测新乘客时也要继续使用这一套参数，**不能重新计算**。

9. 转换成 Tensor，并使用 Dataset 和 DataLoader

数据经过 Pandas 处理后，需要转换成 PyTorch 能够直接使用的 Tensor。训练集的特征和标签分别转换成 float32 类型。

接下来使用 TensorDataset 把每一条乘客数据和对应的 Survived 标签**绑定**起来，再使用 DataLoader 按照 batch_size=32 分成一个个**小批次**。训练集 DataLoader 设置了 shuffle=True，表示**每一轮训练时都会打乱**训练数据的顺序。测试集不需要学习，所以设置为 shuffle=False。

10. 定义神经网络模型

模型使用 PyTorch 的 nn.Module 定义。网络一共有**三个全连接层**，中间加入**两个 ReLU** 激活函数。输入层的大小由经过预处理以后**实际得到的特征数量决定**，然后依次经过 16 个神经元、8 个神经元，最后输出 1 个数。

我的小想法：这里没有直接把输入特征数量写成一个固定数字，而是让程序根据训练数据自动获取。因为前面进行了独热编码，如果分类变量的类别发生变化，最终特征数量也可能发生变化，所以自动读取输入维度会更加方便。

最后一层只有一个输出，是因为这是二分类问题。

11. 选择损失函数和优化器

这个实验的目标只有生还和未生还两种结果，因此使用 `BCEWithLogitsLoss` 作为二分类损失函数。

优化器选择 Adam，学习率设置为 0.001。

12. 进行小批量训练

正式训练时，程序最多训练 100 个 epoch。在每个 epoch 中，对于每个 batch，模型先根据输入数据进行前向计算，得到预测结果，然后计算损失。接着清空之前留下的梯度，再通过反向传播计算当前损失对应的梯度，最后让 Adam 优化器根据梯度更新模型参数。这个过程不断重复，模型就会逐渐调整自己的参数。

程序同时记录每一轮的训练 Loss、训练准确率和测试准确率，这些数据后面会用来绘制曲线。

13. 计算预测概率和准确率

模型最后输出的并不是直接的 0 或 1，而是一个数值。我在计算预测结果时使用 Sigmoid，把这个数转换成 0 到 1 之间的概率。

我使用 0.5 作为分类阈值，大于等于 0.5 就预测为 1，小于 0.5 就预测为 0。然后把预测结果和真实标签进行比较，就可以计算当前训练集或测试集的准确率。

14. 保存表现最好的模型（我的小巧思，我觉得这样有利于模型更高效的学习）

训练过程中，程序会一直比较当前测试准确率和之前保存的最高测试准确率。如果当前结果更高，就把当前模型保存下来，文件名为 `titanic_model_best.pth`。

这个文件不只是保存神经网络参数，还保存了后面预测新样本所需要的重要信息，例如输入特征数量、Age 的填充值、Embarked 的填充值、训练集的平均值和标准差、特征列顺序以及分类变量的类别映射。

这样做的好处是训练结束以后，`predict.py` 不需要重新猜测训练时是怎样处理数据的，而是直接使用保存下来的处理规则。

15. Early Stopping（我的小巧思，训练 100 轮时可能觉得意义不大，但是成千上万轮时，就可以节省很多算力资源）

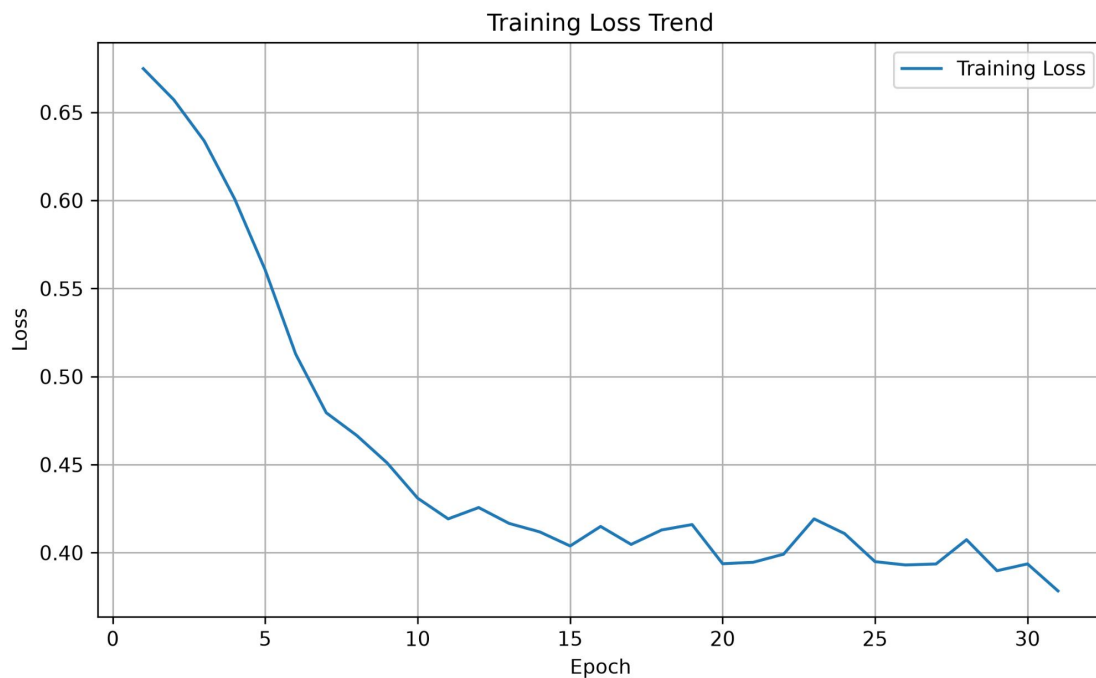
我还加入了 Early Stopping，也就是提前停止训练。虽然最大训练轮数设置为 100，但如果测试准确率连续 20 轮都没有提高，程序就会提前停止。

这样做主要是为了避免模型一直进行没有明显效果的训练，同时也可以节省时间。

16. 绘制训练曲线

训练完成以后，程序把之前每一轮记录下来的 Loss 和 Accuracy 画成两张图。

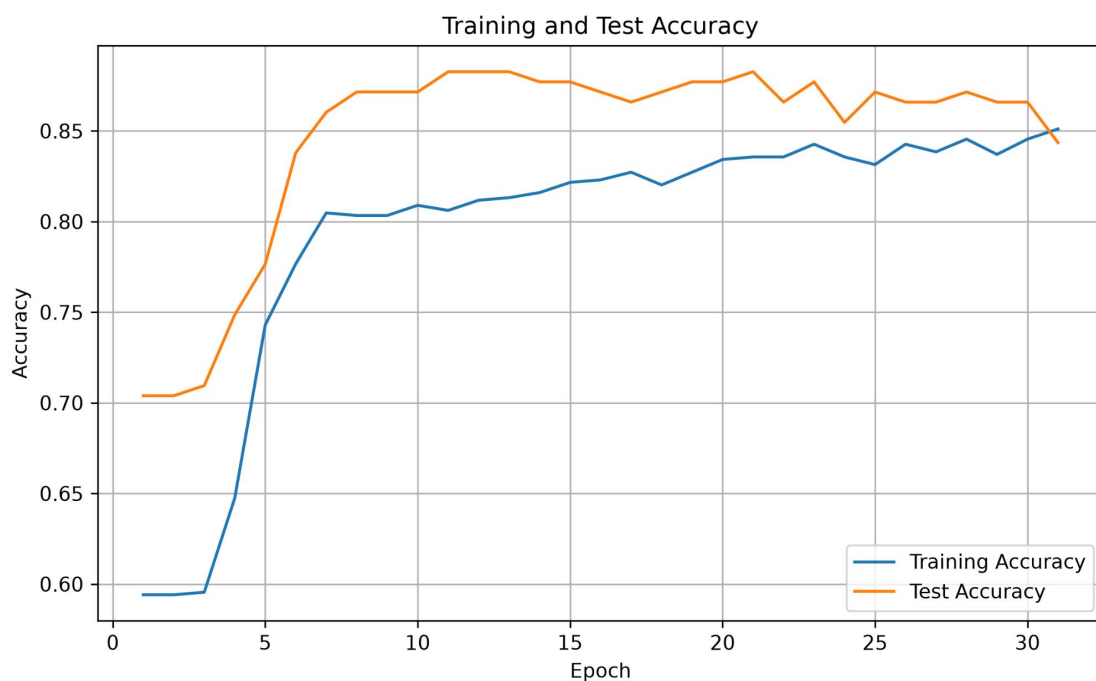
(1) 训练 Loss 趋势图



从这张图可以看出，训练 Loss 在前面的训练阶段下降比较明显，后面的变化速度明显变慢。虽然中间存在一些小幅度的上下波动，但是整体趋势仍然是在下降。

这说明模型在训练过程中确实在逐渐调整参数，使自己的预测结果和训练数据中的真实标签越来越接近。后期 Loss 的下降幅度变小，也说明模型已经学到了一部分比较稳定的规律。

(2) 训练集与测试集准确率



这张图同时显示训练集准确率和测试集准确率。可以看到，训练初期训练准确率从比较低的水平快速上升，之后逐渐变得平稳。测试准确率同样在前期快速提高，之后主要在较高水平附近上下波动。

从后半段可以看到，测试准确率已经没有持续上升，而是出现一定波动。这也是我设置 Early Stopping 并保存最佳模型的一个原因：不能简单地认为训练轮数越多，模型在测试数据上的表现就一定越好。

训练中的小插曲

一开始我用的其实是 1000 轮，但我发现准确率在 100 轮之后就一直在较高水平上下波动，没有明显的提高。甚至还出现了训练集准确率高于测试集的现象，我觉得此时模型已经出现了过拟合的危机，所以我果断减少了训练轮次。

三、predict.py：使用训练好的模型预测新乘客

1. 重新建立模型并加载训练结果

训练完成以后，predict.py 不需要再次训练模型，而是直接读取 trainiTrain.py 保存的 titanic_model_best.pth。

由于 `.pth` 文件主要保存的是训练好的参数和相关信息，所以 `predict.py` 中仍然需要定义和训练时相同的 `TitanicClassifier` 网络结构。网络结构重新建立以后，再把保存的参数加载进去。最后调用 `eval()`，让模型进入预测模式。

同时，程序还从模型文件中读取训练阶段保存的预处理信息。因为预测新乘客时必须按照训练模型时完全相同的方法处理数据。

2. 输入新乘客的信息

我在 `predict.py` 设计了一个 `predict_passenger` 函数，用来接收一名新乘客的信息。

得到这些原始信息以后，程序会再次计算 `FamilySize`、`IsAlone` 和 `FarePerPerson`。这样新乘客的特征构造方式就和训练数据完全一致。

3. 对新乘客进行缺失值处理

如果新乘客没有填写年龄，程序不会重新根据这个乘客的信息计算一个新的年龄中位数，而是直接使用训练阶段保存的 `Age` 中位数。`Embarked` 缺失时也是同样的处理方式。

这样可以保证训练数据和预测数据遵循同一套规则。

4. 对分类信息进行相同的编码

`Sex`、`Embarked`、`Deck` 和 `Title` 在预测阶段也需要转换成独热编码。但是 `predict.py` 不能随意重新创建一套类别，而是使用训练阶段保存的 `category_maps`。

完成独热编码以后，还要按照训练阶段保存的 `feature_columns` 调整列顺序。如果某个特征在新乘客中没有出现，就补成 0。

5. 使用训练阶段的标准化参数

新乘客的数值特征还要进行标准化。这里使用的是训练阶段保存的平均值和标准差，而不是根据新乘客重新计算。

因为模型在训练时看到的是标准化后的数据，所以预测时也必须把新数据转换到同样的尺度。

6. 送入模型并得到预测结果

所有预处理完成以后，新乘客的数据会被转换成 `Tensor`，然后送入训练好的神经网络。预测阶段使用 `torch.no_grad()`，因为这时候只需要得到结果，不需要计算梯度，也不需要更新模型参数。

模型输出经过 Sigmoid 后得到一个 0 到 1 之间的数，这里可以理解为模型预测该乘客生还的概率。例如输出约为 0.80，就表示模型当前认为该乘客属于“生还”类别的概率约为 80%。

7. 交互式预测（这是我觉得最有意思的地方，因此我也在这里花费了大量精力）

为了方便实际使用，predict.py 还设计了交互式输入。运行程序以后，它会依次询问乘客的舱位、性别、年龄、家庭人数相关信息、票价、登船港口、甲板和称谓。

程序对这些输入进行了基本检查，例如舱位只能输入 1、2、3，登船港口使用 C、Q、S 等。如果输入不符合要求，就会提示重新输入，这样可以减少因为输入格式错误导致程序直接报错的情况。（降低程序崩溃的概率）

最后 interactive_predict() 调用预测函数，并把模型预测出的生还概率和最终的 0/1 结果显示出来。

四、实验结果与学习总结

通过这次实验，我把数据处理、神经网络训练和实际预测三个部分连接了起来。刚开始看这个项目时，我觉得重点只是定义一个神经网络，但真正做下来后才发现，数据预处理同样占了很大一部分工作，以及交互式预测的部分。

另外，我也明白了训练完成以后还要保存预处理参数的重要性。模型本身只知道自己训练过的数字特征，并不知道新的乘客数据应该怎样处理。因此 predict.py 不仅要加载模型参数，还要加载训练阶段得到的缺失值参数、标准化参数、分类类别和特征列顺序。

总体来说，这次实验让我对一个比较完整的 PyTorch 二分类项目有了更具体的认识，也把之前学到的数据集、特征工程、DataLoader、损失函数、优化器、反向传播和模型预测等知识联系了起来。

五、项目文件结构

```
LR/
├─ src/
│   ├─ tainiTrain.py
│   ├─ predict.py
│   ├─ train.csv
│   └─ titanic_model_best.pth
```

```
|   └─ requirements.txt
└─ docs/
    └─ loss_curve.png
    └─ accuracy_curve.png
    └─ __Titanic_PyTorch 学习笔记.pdf
```